임베디드시스템 1차 발표

장경민, 김준성, 최윤석

INTELLIGENT ROBOTICS
AND COMPUTER VISION LAB

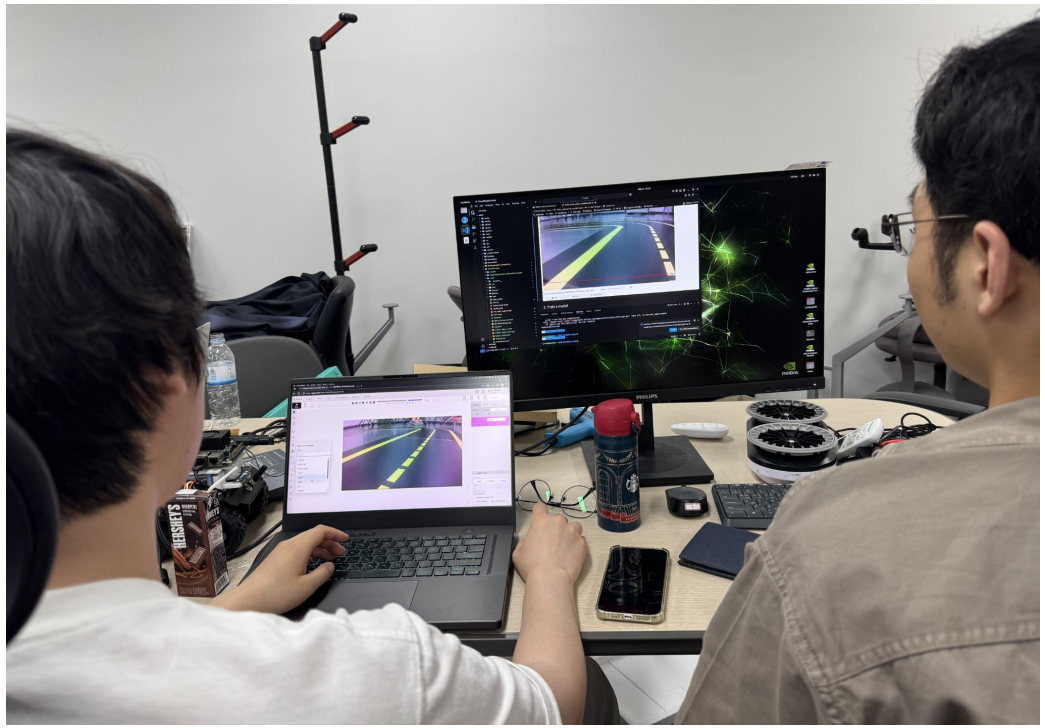
HANYANG UNIVERSITY

Index

- Dataset 제작
- 전체 경로 이동
- Task별 사용한 기능
- 아쉬운 점

Dataset 제작

1. 약 27000장의 frame으로 영상 제작
2. 중앙선을 향해 점을 찍은 후 150번 동안 학습 진행 (소요 시간: 15시간)
3. CVAT 사이트에서 라벨링 진행 후 150번 동안 학습 진행 (소요 시간: 20시간)



Dataset 제작

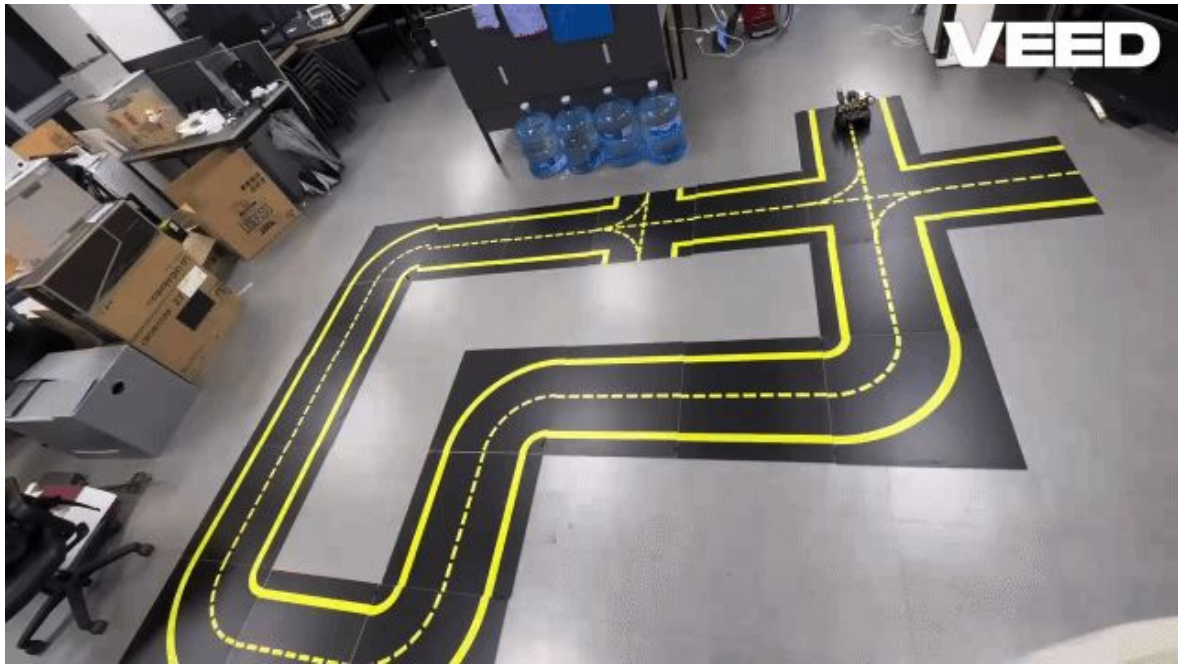
- 실시간으로 찍은 사진들을 영상으로 제작

```
sudo apt update && sudo apt install ffmpeg                                          // ffmpeg 설치

ffmpeg -framerate 30 -i frames/frame_%06d.jpg -c:v libx264 -pix_fmt yuv420p output_video.mp4     // ffmpeg로 동영상 제작
```

- CVAT 사이트에서 라벨링 후 학습 과정은 개인 노트북으로 진행 (gpu 사용에 용이함)

전체 경로 이동

- 중앙선을 중심으로 하는 경로 제작
- 최대한 바깥 선을 벗어나지 않게 하기 위해 PID제어 진행



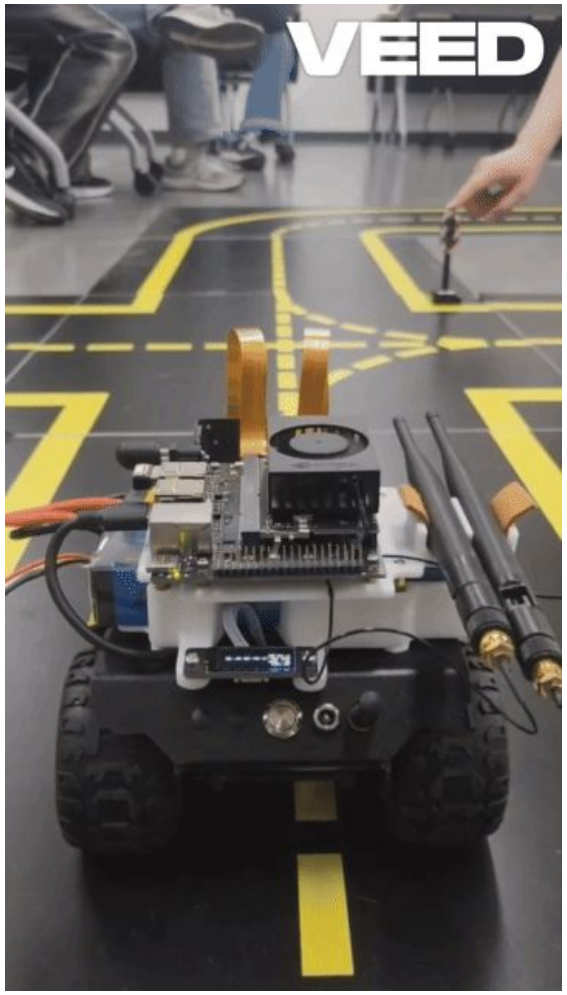

```
# ==================== PID 파라미터 ====================
Kp, Kd, Ki = 1.0, 0.15, 0.1
turn_threshold = 0.7
integral_threshold = 0.2
integral_min, integral_max = -0.4 / Ki, 0.4 / Ki
cruise_speed, slow_speed = 0.45, 0.35
```

```
if abs(err) > integral_threshold or prev_err * err < 0:
    integral = 0.0
else:
    integral = np.clip(integral + err * dt, integral_min, integral_max)

steering = Kp * err + Kd * (err - prev_err) / dt + Ki * integral
prev_err = err

throttle = cruise_speed if abs(steering) < turn_threshold else slow_speed

throttle, steering = handle_task_logic(frame_bgr, throttle, steering)
```

Task 1

- 신호등 빨간 불(red_light)를 인식하면 정지
- red_light이 아닌 경우 전진

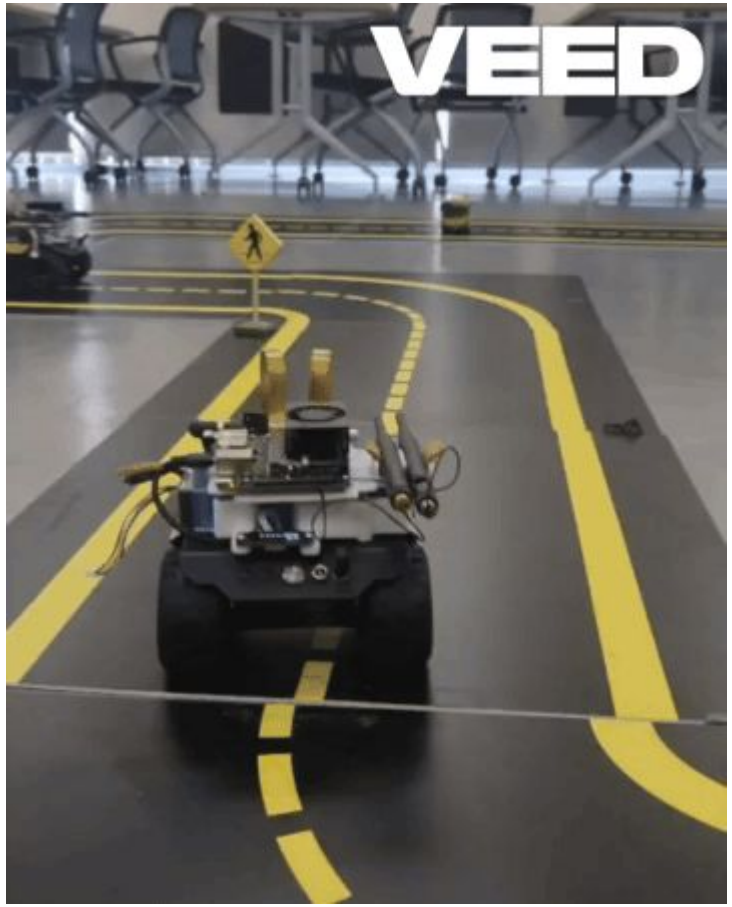
```
if red_lights:
    red_lights.sort(reverse=True, key=lambda x: x[0])
    nearest_area, _ = red_lights[0]
    if nearest_area >= 450:
        print("[TRAFFIC] RED LIGHT → STOP")
        return 0.0, 0.0
```



Task 2

- 일정 거리 내에 SLOW, STOP 표지판을 인식
- SLOW에는 5초 동안 느리게 간 후 다시 전진

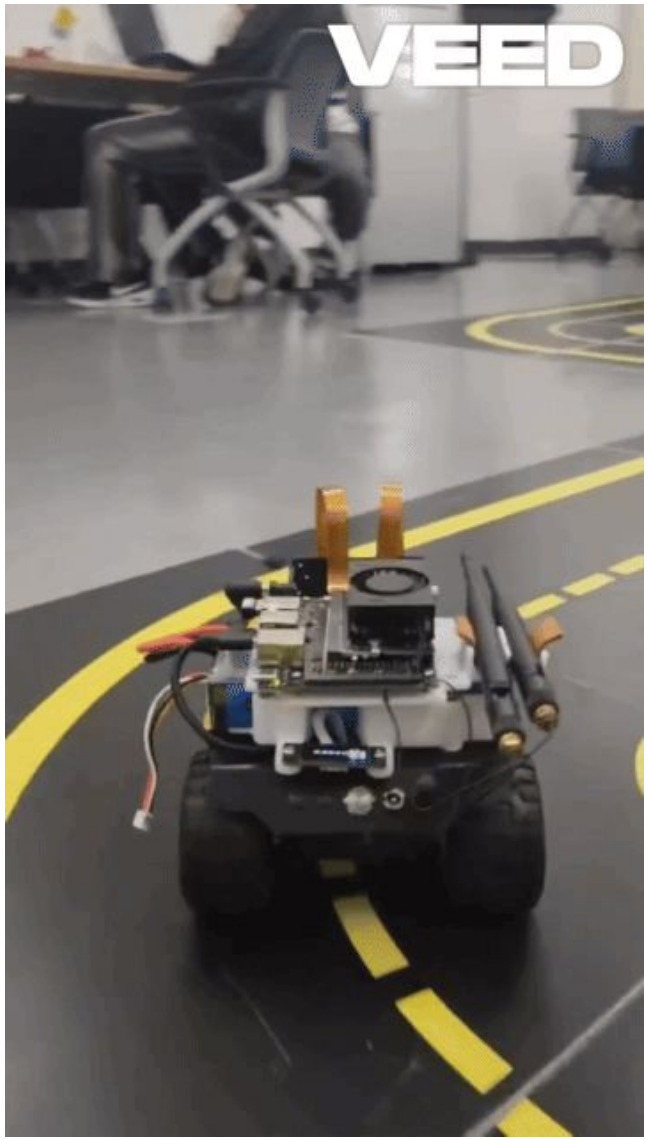
```
if slow_mode:
    if now - slow_start_time < SLOW_DURATION:
        print(f"[STATE] SLOW MODE ({now - slow_start_time:.1f}/{SLOW_DURATION}s)")
        return SLOW_SIGN_SPEED, steering
    else:
        slow_mode = False
        print("[STATE] SLOW MODE → NORMAL")
```



Task 2

- 일정 거리 내에 SLOW, STOP 표지판을 인식
- STOP에는 3초 동안 멈춘 후 다시 전진

```python
if stop_mode:
    if now - stop_start_time < STOP_DURATION:
        print(f"[STATE] STOP MODE ({now - stop_start_time:.1f}/{STOP_DURATION}s)")
        return 0.0, 0.0
    else:
        stop_mode = False
        print("[STATE] STOP MODE → NORMAL")
```



Task 3

- 차량을 인식 후 회피 기동 실행

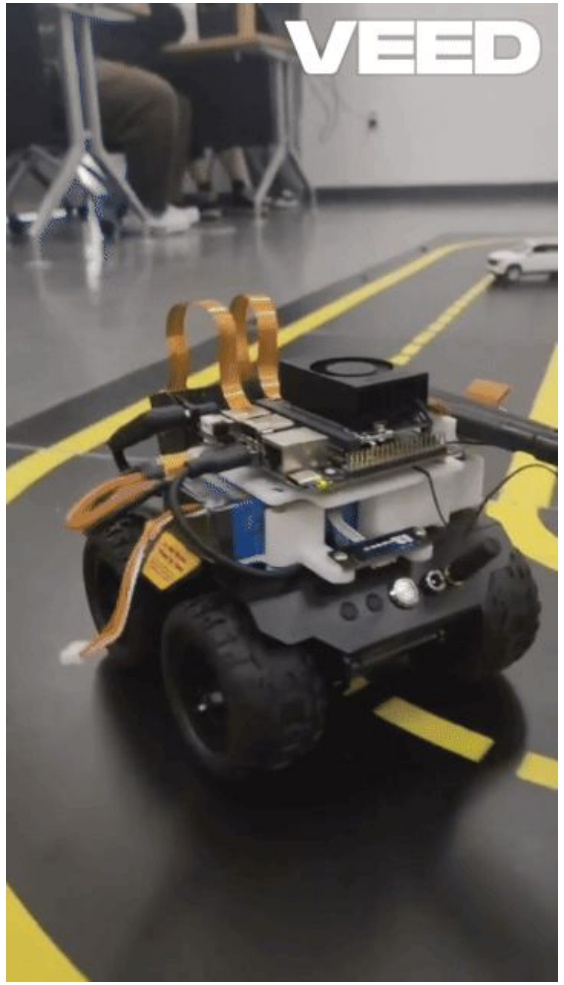
- 3단계로 나눠서 회피 실행

1) Avoiding

2) Straight

3) Recovery

```
if avoidance_state == 'avoiding':
    if avoidance_timer < AVOID_DURATION:
        steering = 0.5 * avoiding_dir
        throttle = 0.25
        print("avoid duration")
    else:
        avoidance_state = 'straight'
        avoidance_timer = 0
```



Task 3

- 차량을 인식 후 회피 기동 실행

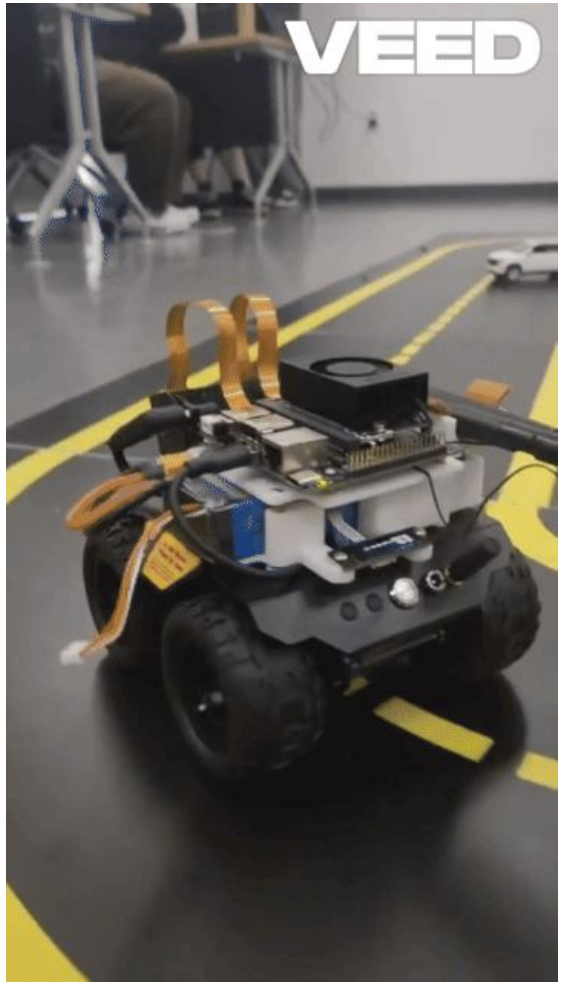
- 3단계로 나눠서 회피 실행

1) Avoiding

2) Straight

3) Recovery

```
elif avoidance_state == 'straight':
    if avoidance_timer < STRAIGHT_ALIGN_DURATION:
        steering = 0.2 * avoiding_dir
        throttle = 0.3
        print("straight align duration")
    elif avoidance_timer < STRAIGHT_DURATION:
        steering = 0.0
        throttle = 0.25
        print("straight duration")
    else:
        avoidance_state = 'recovery'
        avoidance_timer = 0
```



Task 3

- 차량을 인식 후 회피 기동 실행

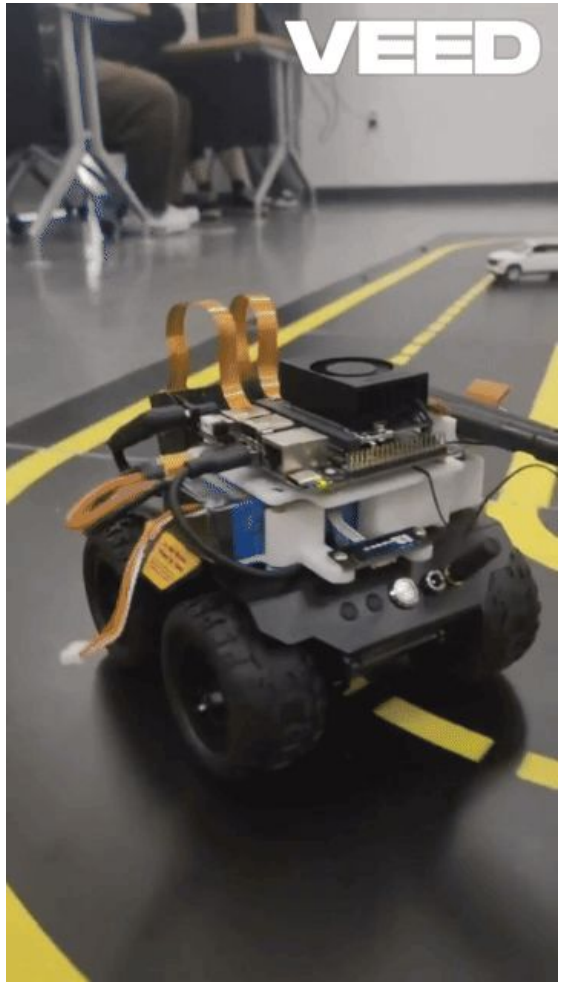
- 3단계로 나눠서 회피 실행

1) Avoiding

2) Straight

3) Recovery

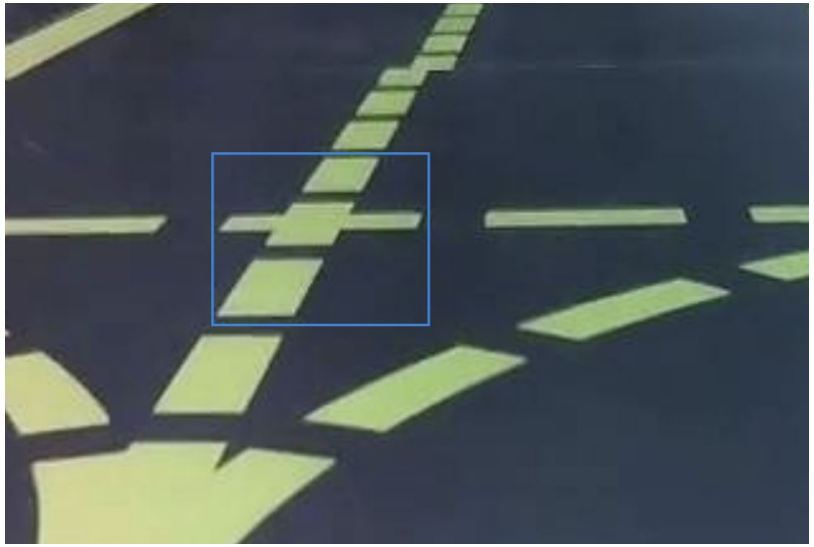
```
elif avoidance_state == 'recovery':
    if avoidance_timer < RECOVERY_DURATION:
        steering = 0.5 * recovery_dir
        throttle = 0.25
        print("recovery duration")
    elif avoidance_timer < RECOVERY_DURATION + RECOVERY_ALIGN_DURATION:
        steering = 0.4 * recovery_dir
        throttle = 0.4
        print("recovery align duration")
    else:
        avoidance_state = 'center'
        avoidance_timer = 0
        avoidance_ignore_counter = AVOID_IGNORE_DURATION
```



Task 4

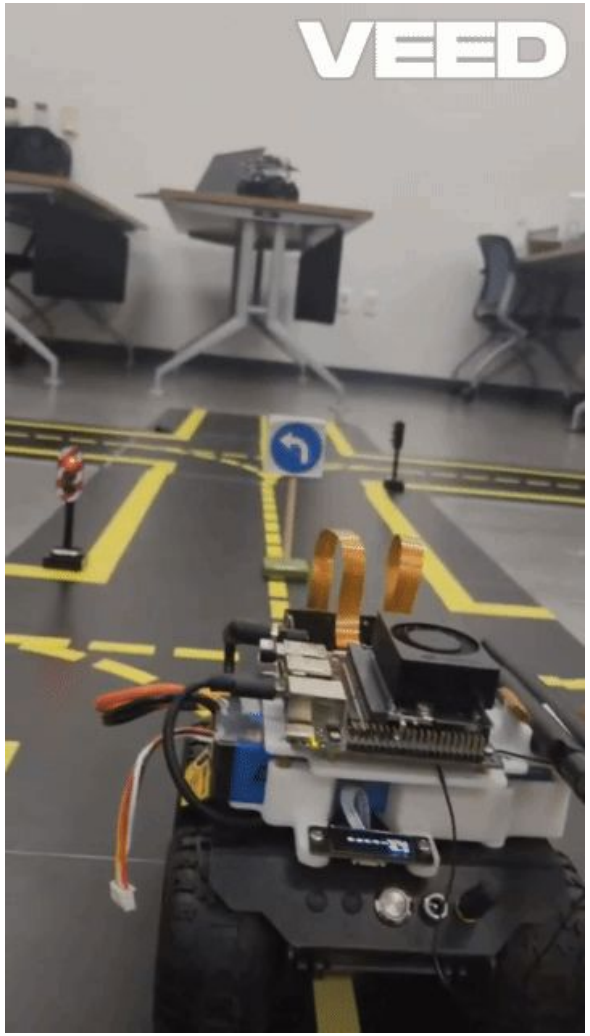
- 신호등 불빛(red_light), 진행 표지판(Left, Right, Straight), 교차로(Intersection)를 동시에 인식
- 가까운 신호등의 red_light이 나오면 우선 정지
- red_light가 꺼진 후, 진행 표지판 인식 및 일정 거리 내 교차로를 인식

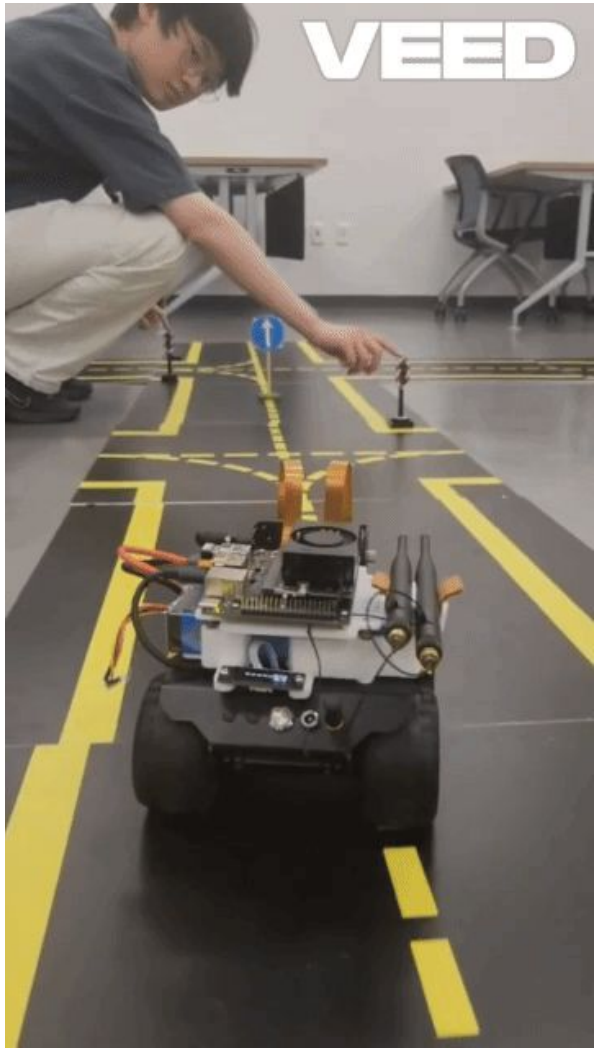
  -> 표지판에 따라 행동(좌회전, 우회전, 직진 중 선택)

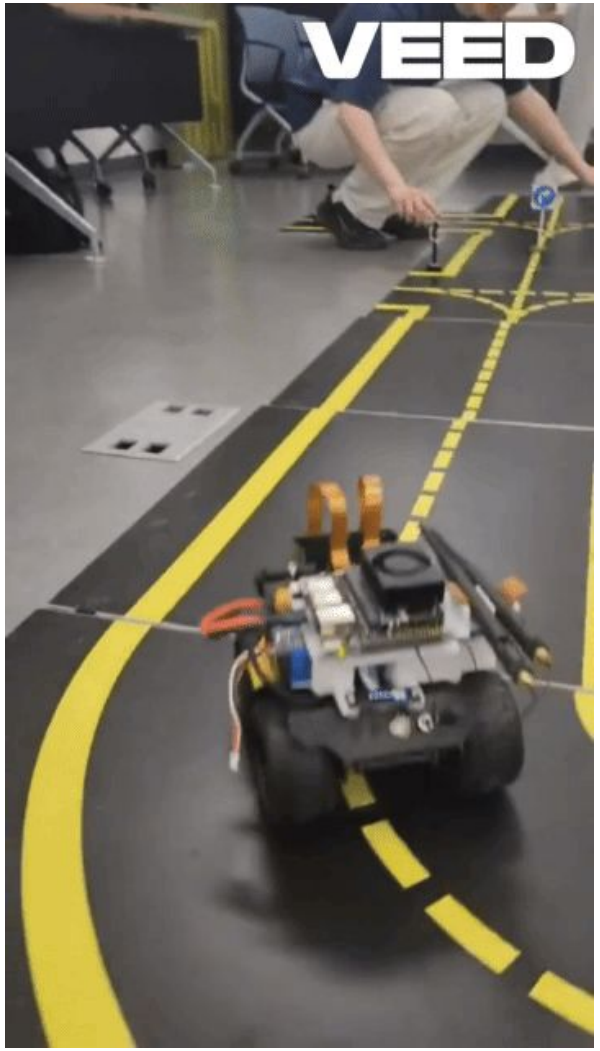


Task 4

동영상 3개 올리는 슬라이드 (좌회전, 우회전, 직진)







기타 고려 사항

- 자연광 유무에 따른 주행 성공도 차이가 큼
  (창문 밖에 햇빛이 생기면, 주행 이탈이 훨씬 더 많이 생김)

- CVAT 사이트를 통해 라벨링 후 학습을 진행해도, 일부 객체가 인식이 안 되는 상황 발생
  (다른 객체들의 조건들을 최대한 이용해서 주행할 수 있도록 설계)

- 관계 없는 객체를 인식하는 경우가 있음
  (빈 공간, 의자, 소화기 등을 Vehicle로 인식)

Thank you!

INTELLIGENT ROBOTICS
AND COMPUTER VISION LAB